Все замеры производились в Docker-контейнере с образом и ограничениями из инструкции (4 ядра cpu, 8гб RAM, 8гб RAM+swap). Железо - Ryzen 7 5800H, 16гб RAM, SSD.

Первоначально было измерено и проанализировано время компиляции при чистой сборке проекта с нуля командами вида:

```
cmake -S "$run_dir/$variant/service_template" \
          -B "$run_dir/build-$variant" \
          -DCMAKE_CXX_COMPILER=/usr/bin/clang++ \
          -DCMAKE_CXX_FLAGS="-ftime-trace" \
          -DCMAKE_BUILD_TYPE=Debug \
          -DCMAKE_EXPORT_COMPILE_COMMANDS=ON \
          -DUSERVER_LTO_CACHE=OFF \
          -DUSERVER_USE_CCACHE=OFF \
          -DUSERVER_COMPILATION_TIME_TRACE=ON
  cmake --build "build-$variant" -j4 --target service_template
```

Также был принят во внимание тот факт, что при реальной итеративной разработке сервиса чаще производится не полная пересборка проекта с нуля, а перекомпиляция только файлов, затронутых правками и зависящих от них. Для симуляции и замеров времени таких частичных сборок после небольших изменений было разработано несколько тестовых сценариев, но об этом ниже.

Уже на первом замере стало видно несколько закономерностей.

Вывод 1. Frontend (парсинг, разбор заголовков и шаблонов) занимает на порядок больше времени, чем backend (кодогенерация и оптимизации). Это означает, что дальше стоит анализировать в первую очередь его. По CBA: 10 компиляций, frontend — 35,2 с, backend — 4,1 с; сумма Total ExecuteCompiler — 39,56 с. Сырой вывод: localresult/clean-baseline-1/cba.txt.

Вывод 2. Самые тяжёлые по времени парсинга файлы и одни из самых долгих по суммарному времени и количеству включений хедеров связаны с proto. Их нет в явном виде в исходниках проекта, они генерируются из файла hello.proto, задающего интерфейс взаимодействия клиента и сервиса. Отсюда появляется гипотеза, что наиболее долгой будет сборка при изменении этого файла, и если такие изменения при разработке сервиса будут происходить часто, то каждая пересборка будет сопоставима по времени с чистой. Стоит присмотреться к такому сценарию и оптимизировать его. По CBA: frontend сгенерированных hello*client.usrv.pb.cpp — 5,29 с, hello*service.usrv.pb.cpp — 5,28 с, hello.grpc.pb.cc — 3,22 с, hello.pb.cc — 2,11 с. Заголовок hello*service.usrv.pb.hpp включался 3 раза (6,01 с суммарно), hello.pb.h — 6 раз (3,81 с). Сырой отчёт: local_result/clean-baseline-1/cba.txt.*

Чтобы просимулировать типичные микрообновления при разработке сервиса, и понять, при каких из них пересборка занимает слишком много времени и может быть оптимизирована, было разработано несколько сценариев замеров, помимо clean - сборки с нуля. В каждом из них вносится незначительное изменение в один из файлов, затем перезапускается и профилируется сборка проекта с теми же флагами, но с переиспользованием незатронутых

изменениями результатов прошлых компиляций: ***proto*** *- вносится изменение в hello.proto, меняется gRPC-контракт взаимодействия сервиса и клиента;* ***header*** *- меняется hello_grpc.hpp - интерфейс обработчика запросов;* ***handler*** *- меняется hello_grpc.cpp - соответственно его реализация, логика обработки запросов;* ***greeting*** *- greeting.cpp - меняется функция, формирующая ответ на запрос;* ***behaviour*** *= greeting+handler - поведение обработчика в целом*

Замеры по самым долгим сценариям показали следующее время (полная таблица - local*result/summary.md):*

Сценарий	Clang, с	Перекомпилированные файлы
clean	39,56	greeting.cpp, hello.cpp, hello.grpc.pb.cc, hello.pb.cc, hello*client.usrv.pb.cpp, hello*grpc.cpp, hello*mongo.cpp, hello*postgres.cpp, hello_service.usrv.pb.cpp, main.cpp
proto	27,28	hello.grpc.pb.cc, hello.pb.cc, hello*client.usrv.pb.cpp, hello*grpc.cpp, hello_service.usrv.pb.cpp, main.cpp
header	8,05	hello_grpc.cpp, main.cpp

Вывод 3. Наиболее долгие сценарии - proto и header. (proto — 27,28 с, header — 8,05 с; для сравнения clean — 39,56 с). Кроме того, файлы, которые перекомпилируются в сценарии proto помимо сгенерированных из hello.proto файлов включают в себя неочевидный main.cpp. Вывод CBA: localresult/proto-baseline-1/cba.txt
Это наблюдение натолкнуло на изучение зависимостей main.cpp от proto-файлов и позволило сформировать гипотезу.

Гипотеза. Время пересборки сервиса в сценарии proto (при изменении файла hello.proto) можно сократить, убрав зависимость main.cpp от генерируемых файлов с помощью реорганизации включений заголовков. В частности, в main.cpp есть включение hello_grpc.hpp, нужное для вызова одного метода: .Append() Здесь используется класс HelloGrpc, и для подстановки в шаблонную функцию Append компилятору требуется его определение, поэтому перекомпиляция hello_grpc.cpp влечёт перекомпиляцию main. Можно создать отдельный маленький заголовок (hello*grpc*registration.hpp), включающий объявление небольшой обёртки над этим методом. Включить его в hello_grpc.cpp, где написать его определение, и в main.cpp, где вызвать эту обёртку. Так main.cpp не будет перекомпилироваться при каждом изменении hello.proto.

Для проверки гипотезы была создана копия сервиса (optimized) с описанными выше изменениями в коде. Были произведены замеры каждого сценария. Первые запуски большинства сценариев показали примерно то же самое время сборки (+- 1 секунда).

Сценарий header радикально ускорился, что неудивительно: main.cpp больше не включает hello*grpc.hpp и потому не перекомпилируется при его изменении. Время сократилось с 8,05 до 3,65 с; вместо двух файлов перекомпилируется один. local_result/summary.md Но основной интерес для проверки гипотезы представляет сценарий proto. Первые замеры показали небольшую разницу 27,28 с у baseline против 22,79 с у optimized, чтобы подтвердить ускорение понадобилось провести дополнительные замеры. Было произведено по 5 запусков версий baseline (изначальной) и optimized в сценарии proto. Результаты следующие: Версия baseline*

показала результаты от 27,24 до 28,02 с, версия optimize - от 21,85 до 27,27 с. Медианы - 27,28 против 22,79 с. Среднее арифметическое - 27,43 против 24,072. Несмотря на то, что самая долгая пятая сборка optimized заняла примерно такое же время, как baseline, среднее арифметическое и медиана времени по всем сборкам показывает, что гипотеза скорее всего верна и действительно немного ускорила сборку в сценарии proto, как и в header.

Подобный приём с сокращением зависимостей при компиляции для её ускорения может быть обобщён и использован на других проектах: если один тяжёлый исходник зависит от другого, то можно связывать их через общий интерфейс (как в данном случае - через маленький хедер с обёрткой над функцией Append). Это позволит в каком-то смысле позволит отложить разрешение зависимости на этап линковки и избежать повторной работы при компиляции. Этот подход имеет достаточно строгие границы применимости, но теоретически может ускорить не только отдельные сценарии сборки, но и полную сборку проекта с нуля - например если таким образом удалось убрать лишнее включение долго обрабатываемого хедера (как в данном случае мы заменили включение hellogrpc.hpp на включение маленького hellogrpc_registration.hpp).